OKR Delivery Report

Practice data. OKR and Jira exports dated 04 November 2026.

How the raw exports became the report

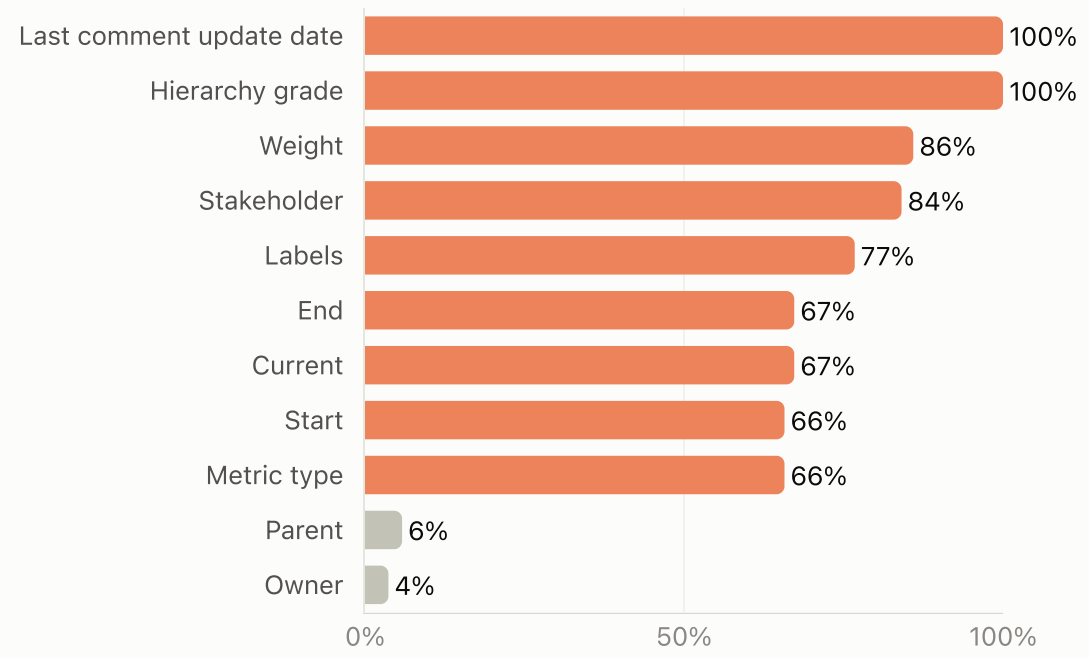
The exploration comes first, and each step shows the evidence, the decision and what it changed. All figures are recalculated from the raw files on every run and cover all cycles unless a step says otherwise.

Step 1

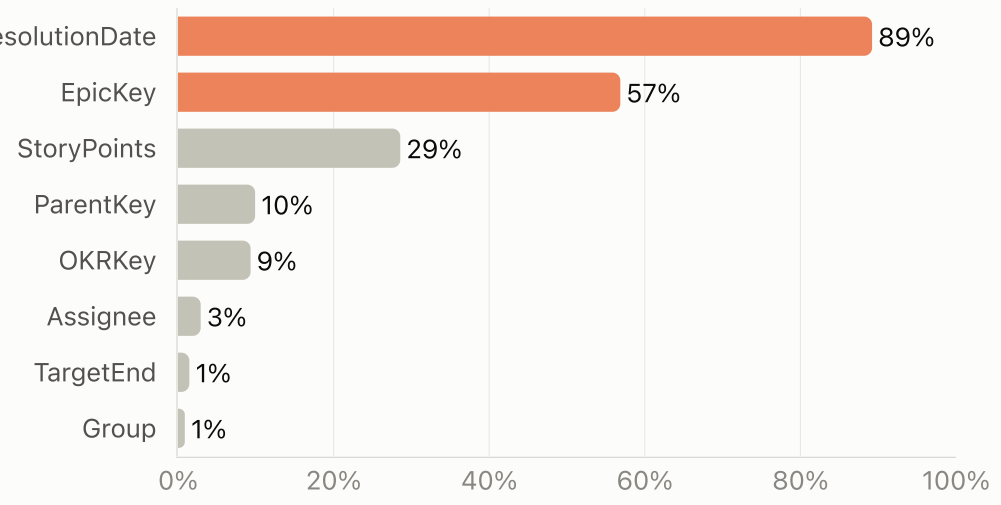
What did we receive, and which columns can be used?

Before any cleaning: how big is each file, and how complete is each column?

OKR file: blank share by column



Jira file: blank share by column



Columns with no blanks are not shown. Orange means at least half the column is blank.

What we found	327 OKRs and 343 Jira issues. Hierarchy grade and Last comment update date are completely empty. Weight is filled on only 46 of 327 OKRs. Metric fields are filled on 112 of 112 Key Results and on 0 other rows, so their blanks are by design. Story points are blank on 0% of Task rows and 50% of Domain/Team rows.
What we decided	Use simple averages rather than weighted ones. Do not build KPIs on the empty columns or on story points.
Why	A weighted score would rest on too few weights to defend. Comparing story points would overstate some issue types, because blanks are not spread evenly across them. A blank is not always a defect, so each was checked before being logged.
Result	All records stay in. The KPIs use only fields that are well filled, or blank for an understood reason.

Step 2

Where does the OKR table actually start?

A standard file read assumes the first row holds the column names. Does it?

Rows above the table

Sheet row	First filled cell
2	OKR Report 04/Nov./2026 - PRACTICE DATA
5	Assignee: none
6	Group: none
7	Average Objective Grade: synthetic

First rows of the table

Sheet row	Column 1	Column 2	Column 3	Column 4	Column 5
10	X-1001 Increase enterprise data di...				
11		X-1002 Digital Channels: Increase ...			
12			X-1003 Increase incident resolutio...		
13				X-1004 Automate evidence collectio...	
14			X-1005 Reduce digital adoption to ...		

What we found	The column names are on sheet row 9, under a title block. The export date is only in the title. The OKR key and title sit together in one of 5 columns, and which column shows how deep the row is in the hierarchy.
What we decided	Find the header by its marker cell ('OKR'). Read the report date from the title. Split the key from the title, and keep the column position as an indent level. Record each row's source row number.
Why	A fixed 'skip N rows' breaks as soon as the title block changes. Dropping the extra columns would lose the indentation, which later repairs the hierarchy (step 7). Reading the date from the title means a new export needs no code change.
Result	327 OKR records, each traceable to its sheet row, and a report date of 04 November 2026 that drives the default cycle and every 'overdue' and 'stale' figure.

Step 3

What is one record, and which key identifies it?

The brief asks for one record per OKR, so we tested each candidate key for repeats.

File	Candidate key	Filled rows	Distinct values	Repeats
OKR file	OKR key	327	327	0
Jira file	Issue key	343	340	3
Jira file	OKR key (where filled)	311	311	0

The repeated issue key

Issue key in file	OKR key on that row	Issue key after repair
DEMO-1023	X-1023	DEMO-1023
DEMO-1023	X-1024	DEMO-1024
DEMO-1023	X-1025	DEMO-1025
DEMO-1023	X-1026	DEMO-1026

What we found	The OKR key has 0 repeats, so the file already holds one row per OKR. The Jira issue key is not unique: 3 repeats, on 4 rows. The pairing DEMO-n to X-n holds on 100% of issues that have no collision.
What we decided	Use the OKR key as the identifier and for the join. Repair a repeated issue key only from that pairing, in a separate column, with a flag. If an OKR key ever repeats, keep the latest row and save the others to okr_duplicates.csv.
Why	A naive drop-duplicates on issue key would have deleted 3 real issues. The repair runs only if the pairing holds on at least 95% of issues, so it switches itself off if the pattern breaks.
Result	327 OKR records from 327 rows (0 duplicates), and every Jira issue kept with a unique final key.

Step 4

Can the dates be read as they are?

We counted the date formats in each column, then tried pandas' default date parser on them.

File	Column	Text month	dd.mm.yyyy	ISO	Blank	Default parser loses
OKR	Start date	320	7	0	0	320
OKR	Due date	327	0	0	0	316
OKR	Last grade update	327	0	0	0	193
Jira	Created	0	0	343	0	0
Jira	Updated	0	0	343	0	0
Jira	Target start	0	6	337	0	6
Jira	Target end	6	0	332	5	6
Jira	Resolution	0	0	37	306	0

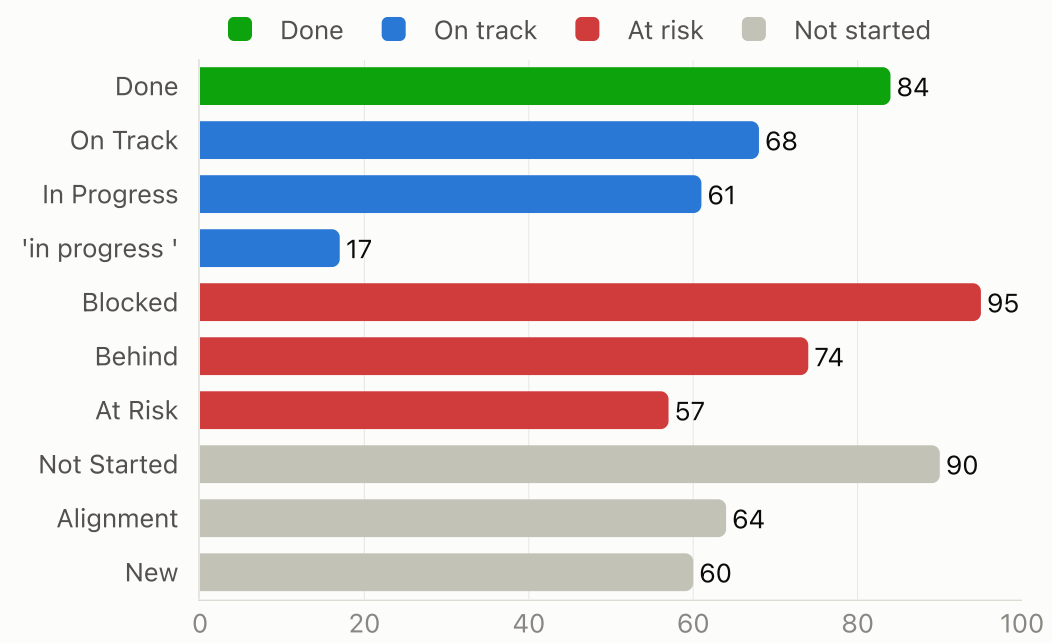
What we found	Each file mixes formats. Month names include Apr., Aug., Dez., Feb., Jan., Juli, Juni, Mai, so the export depends on the language of whoever downloaded it. pandas' default date parser would have silently lost 841 filled dates.
What we decided	Read all text as text first, then parse with one parser that knows each format. Month names live in the settings. A filled date it cannot read stops the run and names the value.
Why	Silent blanks are the worst outcome here: an unread due date just disappears from the overdue count, with no error. A stopped run is annoying, but a wrong number that nobody notices is worse. Formats are matched by pattern, so a day can never be read as a month.
Result	No filled date was lost across either file, and each original value is kept beside the parsed one.

Step 5

Which statuses mean the same thing?

The files use 10 different labels. A chart with that many slices is unreadable, so we asked which ones can honestly be combined.

Each label, coloured by the group it joined



Meaning of each group

- **Done:** Finished. The word is unambiguous, and it is the only status with Jira resolution dates.
- **On track:** Work is moving, with no reported problem. In Progress does not promise health, so this group may flatter delivery slightly.
- **At risk:** Someone has reported a problem with delivery: late (Behind), threatened (At Risk) or stopped (Blocked).
- **Not started:** Delivery has not begun. Alignment is read as still agreeing scope, which is a judgement to confirm with the OKR owner.

Testing the labels against the data

Label as written	Report group	OKR rows	Jira rows	Avg grade (OKR)	Jira rows with a resolution date
Done	Done	38	46	48.3	80%
On Track	On track	33	35	53.5	0%
In Progress	On track	32	29	46.6	0%
'in progress '	On track	9	8	41.9	0%
Blocked	At risk	47	48	46.6	0%
Behind	At risk	34	40	50.0	0%
At Risk	At risk	29	28	47.1	0%
Not Started	Not started	44	46	56.5	0%
Alignment	Not started	32	32	55.6	0%
New	Not started	29	31	50.3	0%

What if a label were grouped differently? (cycle 2026, 165 OKRs)

Label	Now in group	OKRs	Group share now	Group share if it moved out
On Track	On track	16	22%	12%
In Progress	On track	20	22%	10%
Behind	At risk	19	32%	20%
At Risk	At risk	12	32%	24%
Blocked	At risk	21	32%	19%
New	Not started	15	37%	28%
Not Started	Not started	29	37%	19%
Alignment	Not started	17	37%	27%

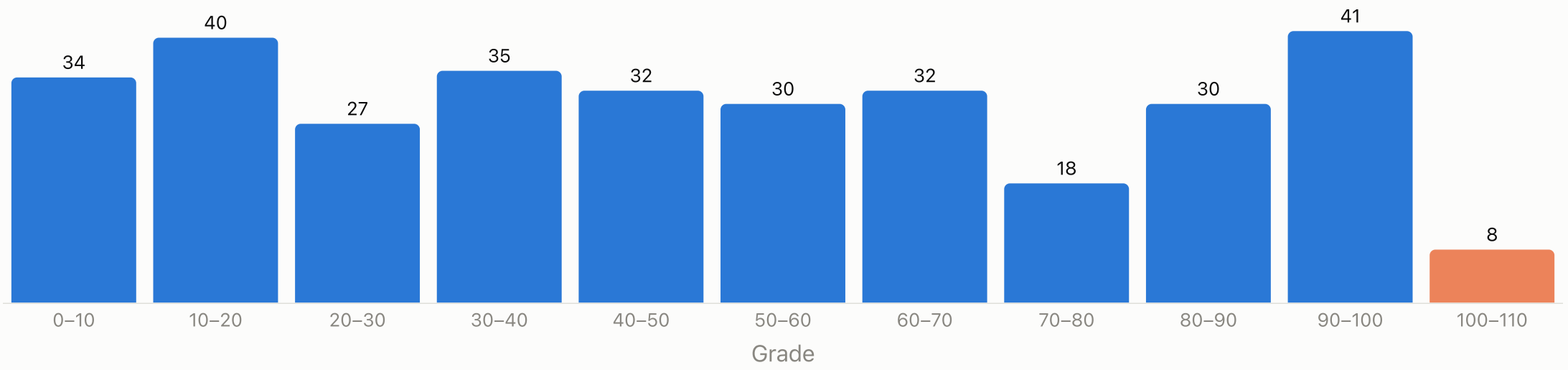
What we found	Four tests. Text: 'in progress ' differs from another label only in case or spacing, which takes 10 labels to 9. Dates: 80% of Done Jira rows have a resolution date against 0% of all others, so Done truly is separate. Grades: they cannot confirm the grouping, since Done averages 48.3 and Not started 54.5, so a higher grade does not mean further along. Team size: with raw labels 85% of team and status combinations have fewer than 5 OKRs; with groups it is 34%.
What we decided	Trim and fix the case first, then map the 9 labels to 4 groups in the settings. Keep the original label in every row. Apply one mapping to both files. A new label stops the run until someone assigns it to a group.
Why	Only the text variants are certain. The rest is a judgement based on what each word means, because grades do not separate the statuses, and the table above shows what each choice costs. The two files agree on status for 307 of 311 linked rows, which is why one mapping serves both. We rejected keeping every label (too thin per team), and guessing a label's meaning from grades (they do not follow status).
Result	The status KPIs and chart use the four groups. The table above is the part to confirm with the OKR owner, since moving one label can change a headline figure.

Step 6

Are the grades on a 0–100 scale, and can they be averaged?

We looked at the full spread of grades, and at whether they follow the Key Result's own progress figures.

How grades are spread across all OKRs



What we found	Grades run from 0.2 to 110, with a median of 48.1. 8 grades sit above 100, all exactly 110, and 6 of them are on work not marked Done. For Key Results, comparing the grade with the progress implied by the start, current and target values gives a correlation of -0.02, and 69 of 107 differ by more than 20 points.
What we decided	Treat 100 as the top of the scale, and say clearly that this is an assumption. Keep the grades above it in the data, flag them, and show the average both ways with 'without' as the main number. Average Key Results only. Warn that grades should not be used to rank teams yet.
Why	Deleting those rows would hide the issue, and capping them at 100 would invent a value. Using them as they are would let a single record lift a small team. Key Results only, because the higher tiers roll up the same work, and including them would count it more than once.
Result	Cycle 2026 Key Result average: 46.5 without (55 Key Results), 47.7 with (56). The largest team effect is Payments Technology in 2027: 54.8 to 45.6.

Step 7

Does each OKR know its parent?

The parent-child structure must survive. We checked the Parent column, then looked for a second source to repair gaps.

Tier by indent level

Tier	Level 0	Level 1	Level 2	Level 3
Domain/Team	0	0	16	0
ERSTE GROUP OBJ.	16	0	0	0
Initiative	0	0	0	151
Key result	0	0	96	16
TRIBE Objectives	0	32	0	0

State of the Parent column

Parent value	OKRs
valid	304
top level	16
placeholder	4
blank	3

What we found	7 OKRs have a blank or placeholder parent (blank 3, placeholder X-99999 4, unknown key 0). The indentation from step 2 gives a second opinion, and it agrees on 304/304 valid parent links. Key Results sit at indent levels 2 and 3, so a tier name does not fix a row's depth.
What we decided	Take the parent from the column where valid. Otherwise take it from the indentation, in a separate column, flagged for the owner to confirm. 'ERSTE GROUP OBJ.' rows have no parent by design.
Why	The two sources agree on 304/304 links we can check, which is why the indentation is trusted where the column is empty. Dropping those rows would have cut off whole branches beneath them. A rule based on tier name alone was rejected, because the tier does not always fix the depth.
Result	7 parents inferred (X-1004, X-1006, X-1008, X-1009…), 0 unresolved, 4 levels in the final tree.

Step 8

What counts as a team?

The brief wants a team breakdown, so we listed every value in the Group column.

Group value as written	OKRs	Reads as
Data Platforms	41	One team
Cloud Engineering	41	One team
Risk Technology	41	One team
Core Services	41	One team
Customer Experience	40	One team
Operations Enablement	39	One team
Digital Channels	38	One team
Payments Technology	38	One team
Digital Channels,Data Platforms	8	Shared by two or more teams

What we found	The column has 9 values but only 8 real teams: 8 OKRs and 7 Jira issues list several teams in one cell. 3 Jira issues have no group.
What we decided	Build a bridge table with one row per OKR and team, used for team views only. Totals are always counted from the OKR table. Blank Jira groups are filled from the linked OKR, and flagged.
Why	Grouping on the raw column would show a fake extra team. Giving a shared OKR to one team would be arbitrary. Splitting it into every team and then summing would double count it, so team rows are not added up to a total.
Result	327 OKRs become 335 team rows, and the totals still count 327. 3 Jira groups filled from the linked OKR.

Step 9

How do the two files connect, and what will not match?

We joined on the key tested in step 3, then gave every unmatched record a reason.

Outcome	Reason	Records
Matched	Linked on OKR key	308
Jira only	No OKR key (label: ad-hoc); no parent either, so a link cannot be inferred	32
OKR only	Expected: the 'ERSTE GROUP OBJ.' tier is not tracked in Jira	16
Matched (issue key repaired)	Issue key was shared with other rows; the link holds because the OKR key is unique	3

What we found	311 Jira rows carry an OKR key, and 311 of those keys exist in the OKR file. All 16 OKR-only rows are the 'ERSTE GROUP OBJ.' tier. The 32 Jira-only rows by type: Task (32); by label: ad-hoc (32).
What we decided	Join on the OKR key, one to one. Keep unmatched records on both sides as named categories and do not force a match. Report unlinked Jira work as its own KPI.
Why	Joining on issue key would break on the repeated keys, and matching on titles would be guesswork. An inner join would have hidden exactly the records the brief asks us to explain. Unlinked work is a finding in its own right: work the strategy cannot see.
Result	359 reconciled records, equal to 327 OKRs plus 32 Jira-only rows. The run stops if these ever fail to add up.

Step 10

Which dates can we trust for time-based KPIs?

The export is dated 04 November 2026. We checked each cycle's dates for sequences that cannot be true.

Cycle	OKRs	Start after export	Jira issues	Updated before created	Created after export	Freshness shown
2026	165	0	174	0	0	Yes
2027	162	162	169	154	169	No

What we found	184 Jira rows carry dates after the export, and 154 were updated before they were created. 118 OKRs in a cycle that has not started already show progress. The Jira updated date equals the OKR last-update date on 311 of 311 linked rows, so those are one source and not two.
What we decided	Default the report to cycle 2026 and label later cycles as planning data. Show freshness only for cycles without impossible dates. Take a linked issue's cycle and due date from its OKR, which fills 5 blank and overrides 5 differing Jira target dates, flagged. Build no cycle-time KPI.
Why	Blending cycles would mix delivery with plans. Cycle time would be the natural Jira KPI, but it needs dates we have just shown to be unreliable. We first drafted freshness from both files as if they confirmed each other, and dropped that when the dates turned out to be identical.
Result	The 90-day freshness KPI is shown for 2026 and labelled as one source. Impossible dates are excluded from overdue counts only; the rows stay.

Step 11

How do we know nothing went wrong?

Every run ends with checks. A failure stops it before any file is overwritten.

Check	This run
Every OKR row is either in the clean table or saved as a duplicate	Passed
One record per OKR key	Passed
Every Jira row is kept	Passed
Reconciliation covers every record	Passed
The report's cycle exists in the data	Passed

What we found	All 5 checks passed. Warnings: none.
What we decided	Separate problems into two kinds. Those that would make totals wrong stop the run. Those that affect a few rows become warnings and log entries.
Why	Stopping on every imperfection would mean the report never runs, while never stopping would let wrong totals reach a manager. Every issue found along the way is counted in the Data quality tab.
Result	327 OKRs and 343 Jira issues in, and the same records out: no record was deleted. Clean tables and the log are in data/processed.

▶ **What the terms mean** New to OKRs? Start here.

Average Key Result grade
46.5
Out of 100, leaving out grades above 100 (with them: 47.7). 55 Key Results.

OKRs at risk
32%
52 of 165 OKRs are Behind, At Risk or Blocked.

OKRs not started
37%
61 of 165 OKRs are New, Not Started or Alignment.

Not updated in 90+ days
74%
122 of 165 OKRs had no grade update in the 90 days before the export.

Jira issues with no OKR
10%
17 of 174 work items are not linked to any goal. 14 Jira items in this view are overdue.

What to act on in 2026

Three findings for all teams this year, each with a suggested action. They do not change with the filters.

37% of 2026 OKRs have not started, and 84% of the year has gone.

61 of 165 OKRs are New, Not Started or Alignment. Only 16 (10%) are finished. Payments Technology has the most not started (55% of its OKRs).

Action: Group leads should decide this month, for each one, whether to re-plan it, shrink it or close it, so the year-end result is not a surprise.

40 of the 52 OKRs at risk have not been updated in 90+ days.

That includes 17 of the 21 that are Blocked. Customer Experience has the most at risk (48%), and Risk Technology the most not updated (95%). 2 OKRs at risk have no owner.

Action: Start with the Blocked ones nobody has updated: give each an owner, and ask for a monthly update on anything at risk.

17 Jira issues (10%) are not linked to any OKR, and 14 of them are overdue.

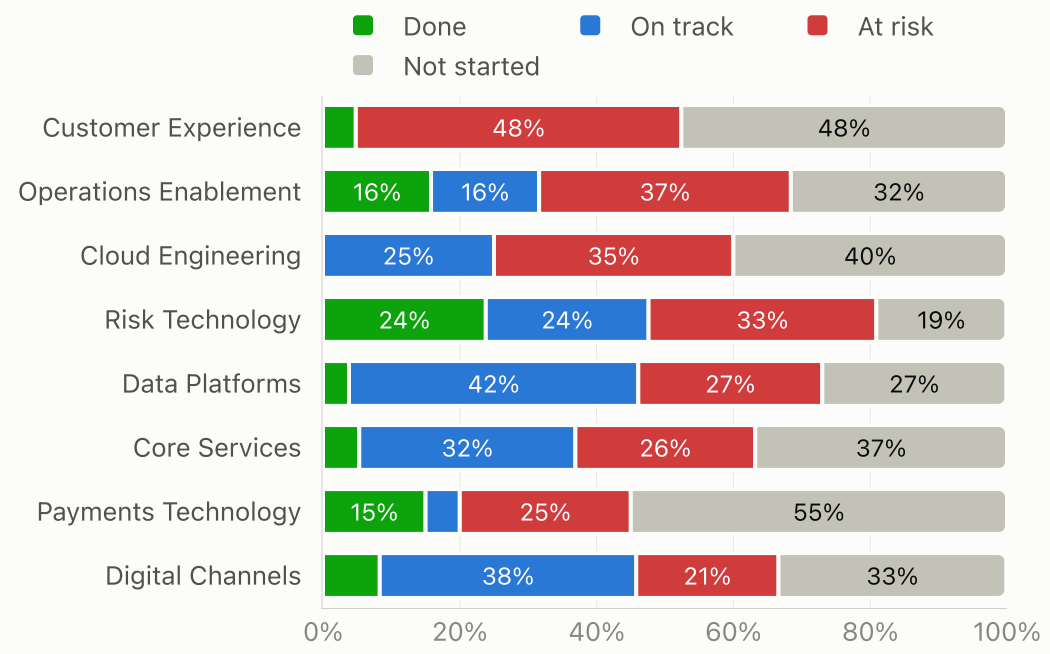
All 14 overdue issues this year are in this group. Data Platforms has the most (4). Note: 153 linked issues are not due yet, so they cannot be overdue yet.

Action: Team leads should go through each one: link it to an OKR, schedule it, or close it. Then agree how much time unplanned work should take.

Use grades with care: finished OKRs average a lower grade (48.3) than ones not started (54.5). Grades also do not match the Key Results' own start, current and target numbers. Each team has only 6–8 Key Results, so one record can change its position. Do not rank teams on grades until a grading rule is agreed.

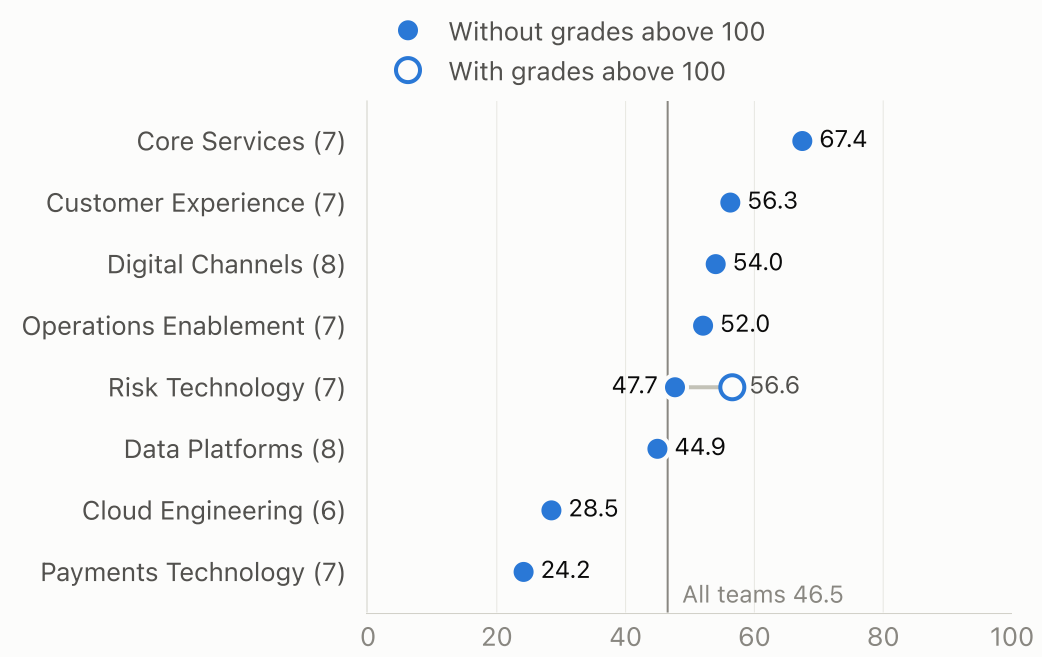
Status mix by team

How each team's OKRs split by status. Teams with the most at risk are at the top.



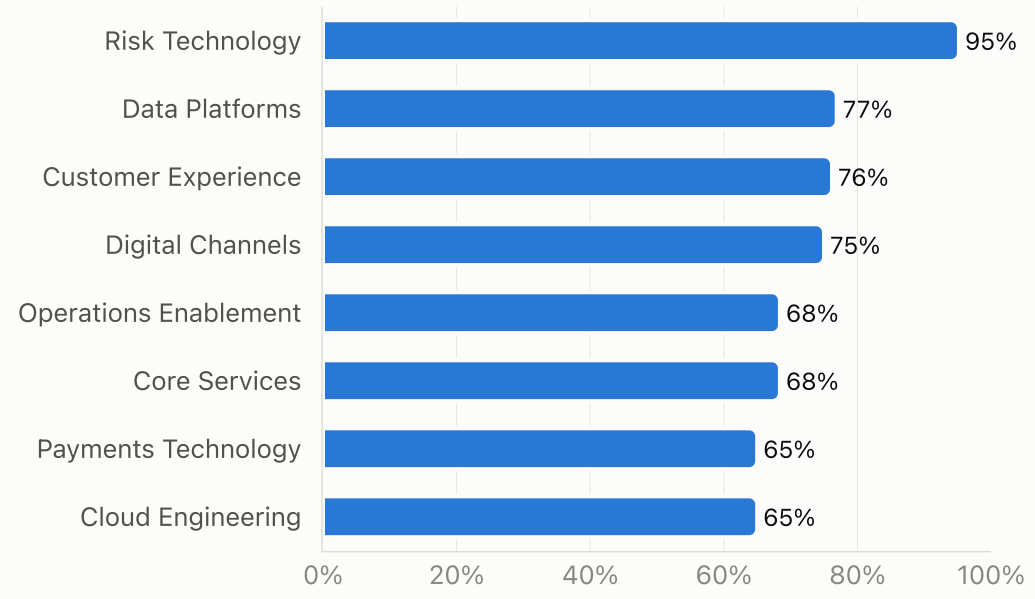
Average Key Result grade by team

Out of 100. Number of Key Results in brackets. Filled dot: the average chosen in the filter. Hollow dot: the other one.



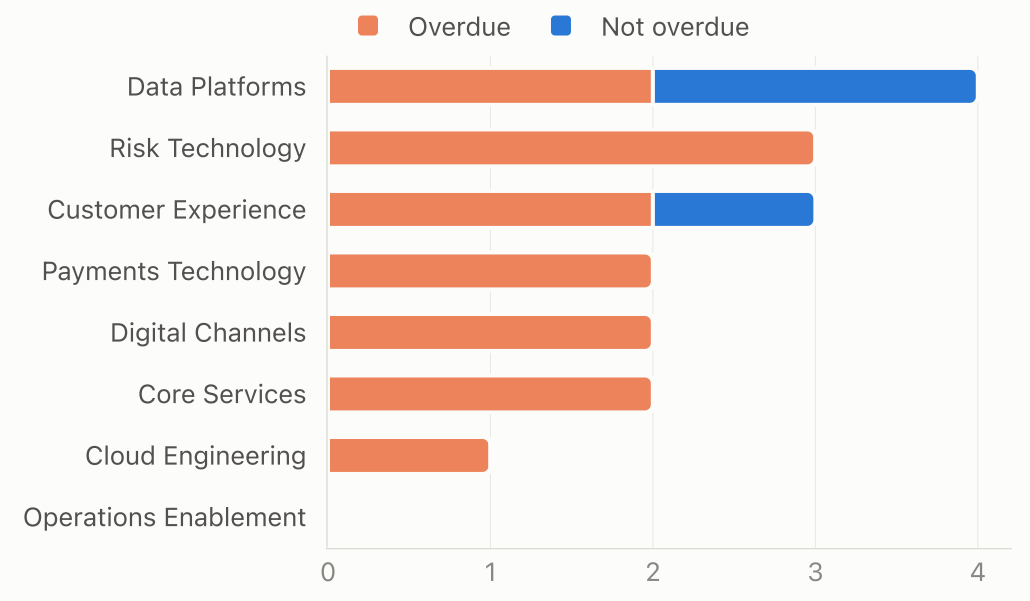
OKRs with no update in 90+ days

Share of each team's OKRs with no grade update in the 90 days before the export. Lower is better.



Jira issues with no OKR link

Work items per team that are not linked to any goal. Orange: past their due date.



Why two grade averages?

The main figure leaves out grades above 100. Across all teams that moves the average by 1.1 points, from 47.7 to 46.5. It matters most for Risk Technology: leaving out X-1087 moves it from 56.6 to 47.7, and from place 2 to place 5. Why: 8 grades are exactly 110, and 6 of them are on work that is not finished. If 100 is the top of the scale these are probably errors, and they push averages up. Nothing is deleted; switch 'Grade average' above to see the other figure.

OKR hierarchy

Every OKR with the goals and Key Results beneath it. Click a row to open it. When a team is selected, grey rows belong to other teams and are shown only to keep the tree whole. Yellow tags point to a data problem.

OKR	Level	Status	Grade	Owner	Updated
▶ **X-1001** Increase enterprise data discoverability in 2026 4 of 19 below are at risk Grade above 100 No owner	ERSTE GROUP OBJ.	In Progress	110.0	Unassigned	293d ago
▶ **X-1021** Accelerate enterprise automated controls in 2026 6 of 21 below are at risk	ERSTE GROUP OBJ.	In Progress	5.1	Taylor Chen	40d ago
▶ **X-1043** Improve enterprise data discoverability in 2026 9 of 20 below are at risk	ERSTE GROUP OBJ.	At Risk	45.8	Chris Vale	156d ago
▶ **X-1064** Accelerate enterprise incident resolution in 2026 8 of 20 below are at risk	ERSTE GROUP OBJ.	New	23.3	Robin Lee	248d ago
▶ **X-1085** Standardize enterprise incident resolution in 2026 6 of 20 below are at risk	ERSTE GROUP OBJ.	Behind	24.7	Chris Vale	163d ago
▶ **X-1106** Accelerate enterprise reporting reliability in 2026 6 of 19 below are at risk	ERSTE GROUP OBJ.	At Risk	78.3	Taylor Chen	143d ago
▶ **X-1126** Improve enterprise incident resolution in 2026 5 of 19 below are at risk	ERSTE GROUP OBJ.	New	57.5	Chris Vale	64d ago
▶ **X-1146** Simplify enterprise digital adoption in 2026 5 of 19 below are at risk No owner	ERSTE GROUP OBJ.	In Progress	37.5	Unassigned	49d ago

Data-quality log

19 problems found in the two files, most serious first. Nothing was deleted: every problem is flagged, fixed in a separate column, or explained. Numbers cover all years.

High changes a headline number or the hierarchy Medium affects some views Low tidy-up or known gap

Priority	Problem	How many	Why it matters	What we did
High	**Next year's OKRs already show progress** 118 of 162 OKRs for 2027 start after the export date, yet are marked Done, Blocked, In Progress and so on.	118 of 162	Mixed in with this year, they would distort the picture of current delivery.	The report opens on 2026 and shows later years separately, as plans. Ask the OKR owner what these statuses mean.
High	**Jira dates in the future** 184 issues were created, updated or finished after the date the file was exported, which cannot have happened yet.	184 of 343	We cannot measure how long work takes or when it was finished.	Kept and flagged. No 'time to finish' measure is built. In 2026 only the finish date is affected; ask whether it is filled in automatically with the planned end date.
High	**Issues updated before they were created** 154 issues, all in 2027, have a last-updated date earlier than their created date.	154 of 343	The update dates cannot be trusted for those years. Jira's updated date is also a copy of the OKR file's, so it cannot be used to double-check it.	'Not updated in 90+ days' is shown only for 2026. Ask how the export produces these dates.
High	**Grades above 100** 8 grades are exactly 110, and 6 of them are on work that is not finished.	8 of 327	If 100 is the top of the scale these are probably errors, and they push averages up.	Kept. The main average leaves them out and the average with them is shown beside it. Ask the OKR owner whether grades above 100 are allowed.
High	**Status and grade disagree** 20 finished OKRs have a grade under 50, and 57 OKRs that have not started have 50 or more.	77 of 327	Finished OKRs average 48.3 and not-started ones 54.5, so a grade does not show how far along the work is.	Both are shown and never combined. Grades are labelled 'indicative'. The owner should define what a grade means.
High	**Grades do not match the Key Result numbers** Each Key Result has a start, current and target value. For 69 of 107, the progress those numbers imply is more than 20 points away from the grade.	69 of 107	Grades cannot be checked or recalculated from the numbers. Also, 23 have a target that points the wrong way, such as a goal to reduce something with a target above where it started.	The grade is used as given, with a note that it cannot be checked. The OKR tool should validate these fields when they are entered.
High	**Missing parent links** 7 OKRs and 8 Jira issues have a blank or dummy parent.	15 of 670	Without a parent, an item cannot be counted under the objective it belongs to.	OKR parents were filled in from the file's indentation, which matched every one of the 304 links we could check. They are marked 'inferred' for owners to confirm. Jira parents are not used in the report.
Medium	**Two teams in one cell** 8 OKRs and 7 Jira issues name more than one team, such as 'Digital Channels,Data Platforms'.	15 of 670	Read as written, this looks like an extra team, and the real teams lose these items.	Each item is shown under every team it names. Totals still count it once, so team figures do not add up to the total.
Medium	**One Jira key used for different issues** 1 key is used on 4 different issues.	4 of 343	Linking the files by this key would attach work to the wrong goal.	The files are linked by the OKR key instead, which is unique. 3 keys were corrected using the usual pattern (DEMO-n belongs to X-n), and flagged.
Medium	**Jira and the OKR file disagree on status** 4 of 311 linked items, most often Jira 'Done' vs OKR 'Alignment'.	4 of 311	The share of finished work depends on which file you ask.	The report uses the OKR status. The disagreements are listed for owners to fix.
Medium	**Finished, but no finish date** 9 of 46 finished Jira issues have no finish date.	9 of 46	We cannot tell when that work was completed.	Still counted as Done, and flagged. No date is made up.
Medium	**End date before start date** 9 Jira issues and 4 OKRs end before they start.	13 of 670	Overdue counts and durations would be wrong.	Where Jira's end date is blank or differs, the OKR due date is used, and flagged. Items that still end before they start are left out of overdue counts only.
Medium	**Jira work with no OKR** 32 issues have no link to any OKR; all are tasks labelled 'ad-hoc'.	32 of 343	This work is invisible in OKR reporting, so teams look less busy than they are, and nobody can see whether it serves a goal.	Shown as its own group, not forced into a match. It is one of the report's five KPIs.
Medium	**Dates written in different ways** 19 dates are written differently from the rest of their column, such as 31.03.2026. Month names also vary: Apr, Aug, Dez, Feb, Jan, Juli.	19 of 670	A standard date reader can silently blank these dates or mix up day and month.	One reader handles every format. If a date cannot be read, the run stops and names it.
Low	**OKRs with no Jira issue** 16 OKRs have no Jira issue; all of them are top-level company objectives.	16 of 327	Expected for the top level, which is not tracked in Jira.	Reported as 'OKR only'.
Low	**No owner** 12 OKRs and 10 Jira issues have no owner.	22 of 670	Nobody is responsible for acting on them.	Shown as 'Unassigned' so the team lead can pick them up.
Low	**No team on a Jira issue** 3 Jira issues have no team.	3 of 343	They would drop out of every team view.	The team is copied from the linked OKR, and flagged.

Priority	Problem	How many	Why it matters	What we did
Low	**The same status written differently** 17 rows have extra spaces or different capitals, such as 'in progress '.	17 of 670	One status would show up as two in the charts.	Spaces and capitals are tidied up. The original text is kept.
Low	**Mostly empty columns** Weight is filled in on only 46 of 327 OKRs, and two other columns are completely empty.	281 of 327	A weighted average would rest on too few weights to be fair.	Simple averages are used, and the report says so. Weights on Key Results should be made mandatory.

Method and assumptions

How the average grade is worked out

- **Only Key Results are averaged.** They are the measurable targets. The goals above them summarise the same work, so including them would count it twice.
- **Every Key Result counts the same.** Only 46 of 327 OKRs have a weight, which is too few to use.
- **Grades above 100 are left out of the main number** and shown beside it. That 100 is the top of the scale is our assumption; the files do not say.
- **An OKR shared by two teams counts for both.** So team figures do not add up to the total, which counts each OKR once.
- **The Contribute column is not used,** because the files do not explain what it means.

Where leaving out grades above 100 changes the result

Every change of 1 point or more. The grades are still in the data.

Year	Level	Team	Average with	Average without	Change	Place with	Place without	Grades left out
2026	Key results	Risk Technology	56.57	47.67	-8.90	2	5	X-1087
2026	Key results	All teams	47.66	46.53	-1.13			X-1087
2026	All OKR levels	All teams	50.28	48.80	-1.48			X-1001, X-1044, X-1087, X-1130
2027	Key results	Payments Technology	54.77	45.57	-9.20	2	4	X-1302
2027	Key results	All teams	47.31	46.17	-1.14			X-1302
2027	All OKR levels	All teams	50.29	48.78	-1.51			X-1173, X-1216, X-1259, X-1302

How statuses are grouped

Group in this report	Statuses in the files
Done	Done
On track	On Track, In Progress
At risk	Behind, At Risk, Blocked
Not started	New, Not Started, Alignment

How the two files were matched

Each Jira issue is matched to its OKR by the OKR key, which is unique. Every record ends up in one of these groups.

Result	Reason	Records
Matched	Linked on OKR key	308
Jira only	No OKR key (label: ad-hoc); no parent either, so a link cannot be inferred	32
OKR only	Expected: the 'ERSTE GROUP OBJ.' tier is not tracked in Jira	16
Matched (issue key repaired)	Issue key was shared with other rows; the link holds because the OKR key is unique	3

Parent links

Where an OKR's parent was blank or a dummy value, we used the file's indentation instead. It agreed with every one of the 304 parent links we could check. These are tagged for owners to confirm.

What this report cannot tell you

- Whether work is being updated rests on one source: Jira's updated date matches the OKR file's on 311 of 311 linked items. It is shown only for 2026, where the dates make sense.
- How long work takes: 36 of 37 finish dates in Jira are after the export date, so this cannot be measured.
- How much effort work takes: story points (effort estimates) are missing on 32% of linked issues but 0% of unlinked ones, so totals would not compare fairly.
- Whether things are improving: this is one snapshot, so it shows where things stand, not the trend.

Source and tools

327 OKRs and 343 Jira issues from Candidate_OKR_Raw_Export.xlsx and Candidate_Jira_Issues.csv. Built in Python (pandas and Plotly). The business rules are in config/settings.yaml. AI use: Claude Code (Anthropic) helped with data profiling, code and drafting.

Automation idea

If both exports are updated regularly, the report can refresh itself. Nobody should need to copy, clean or paste anything by hand, and a bad file should never reach a manager.

1. New exports arrive Saved to a shared folder, or pulled from Jira on a schedule. → **2. Check the files** Columns, statuses and dates are compared with the rules file. → **3. Clean and link** The same steps as the Data preparation tab, every time. → **4. Check the totals** 5 checks, such as "no record lost" and "one row per OKR". → **5. Publish** Report, PDF and data-quality log are updated; owners get a list of new problems.

If a check fails: stop, publish nothing, keep the last good report online, and alert the owner with what to fix.

Which steps to automate

- **Collecting the files**, so there is no manual download and upload.
- **Cleaning, linking and checking**, so the same rules are applied the same way every time.
- **Rebuilding the report and the data-quality log**, and sending owners the problems that concern them.

How the data stays accurate

- **One rules file.** Status groups, thresholds and column names live in one place (config/settings.yaml), so every run uses the same rules and a change is made once.
- **The run checks itself.** If any of the 5 total checks fails, nothing is published.
- **Problems stay visible.** The 21 data-quality checks run every time. Comparing their counts with the last run would show when something gets worse, such as more grades above 100.
- **Rule changes are tested first,** including against deliberately broken files.

What happens if something goes wrong

- **A file has changed** (a renamed column, a new status or an unknown date format): the run stops and names the problem. The last good report stays online. Someone updates the rules file and runs it again.
- **A few rows are wrong** (a missing parent, an impossible date): the report is still published, the rows are flagged in the data-quality log, and owners get a list to fix at the source.
- **A file is missing or late:** the report always shows the export date, so readers can see how old it is. If no new file arrives on time, the owner is alerted.
- **A wrong number is found after publishing:** each run keeps its input files, so the earlier version can be restored and the difference explained.

What exists today and what would be added

Part	Status
Rules in one settings file	Built in this project
Checks that stop the run before anything is written	Built in this project
Last good output kept when a run fails	Built in this project
Data-quality log rebuilt on every run	Built in this project
Tests with deliberately changed files	Built in this project
Running on a schedule and fetching files automatically	To add
Alerts to owners when a run stops or a file is late	To add
Keeping each run's inputs and comparing with the last run	To add